



OBJECT-ORIENTED SYSTEM DESIGN (OOSD)

(BCS-054)

Unit-I : Introduction

Complete Notes

UNIT 1 - SYLLABUS

Introduction: The meaning of Object Orientation, object identity, Encapsulation, information hiding, polymorphism, generosity, importance of modelling, principles of modelling, object oriented modelling, Introduction to UML, conceptual model of the UML, Architecture.

The meaning of Object Orientation

Object-Oriented Programming (OOP) is a programming paradigm in which software is developed using objects, where each object contains data (attributes) and functions (methods). It helps in modular, reusable, and secure program design by modeling real-world entities using classes and objects.

Components of Object Oriented Programming

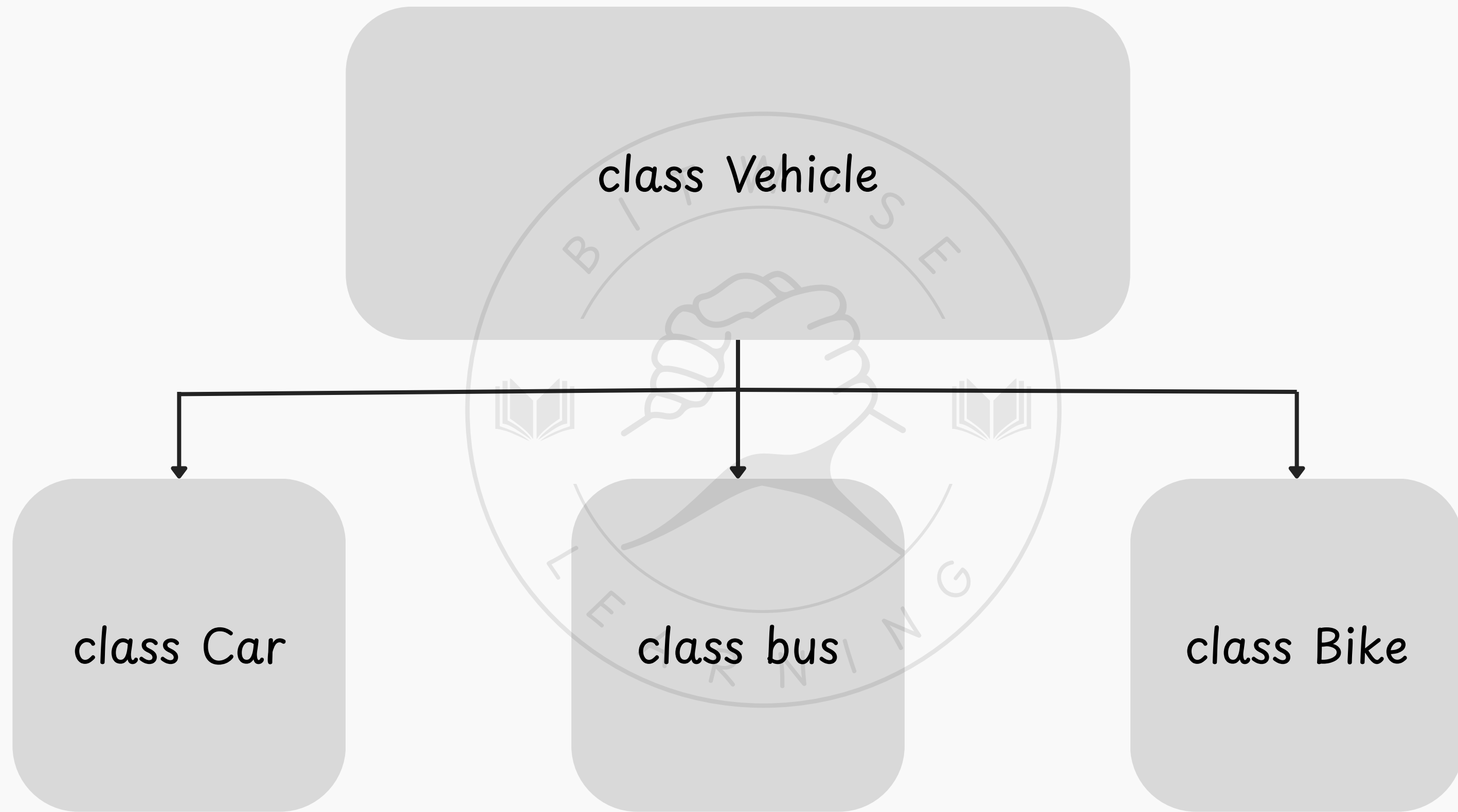
1. **class** - A class is a blueprint or template used to create objects.
2. **object** - An object is an instance of a class that contains data and functions.
3. **encapsulation**- Wrapping data and methods into a single unit to provide data security.

- 4. **abstraction** - Refers to the act of representing essential features, without including background detail or explanation.
- 5. **inheritance** - A mechanism in which one class acquires the properties and methods of another class.
- 6. **polymorphism** - Ability to take more than one form.

Features of Object Oriented Programming

- Programs are divided into objects.
- Objects contain data and functions together.
- Data is secured from unauthorized access.
- Code can be reused using inheritance.

OOSD (BCS-054)



Object Identity

Object identity means every object has its own unique existence in memory, even if two objects contain the same data or values.

Simple Example:

- Two students may have the same name and marks, but both are different persons.
- Similarly, in OOP, two objects with same data are still considered different because each has a unique identity.

Programming Example:

```
Student s1 = new Student();  
Student s2 = new Student();
```

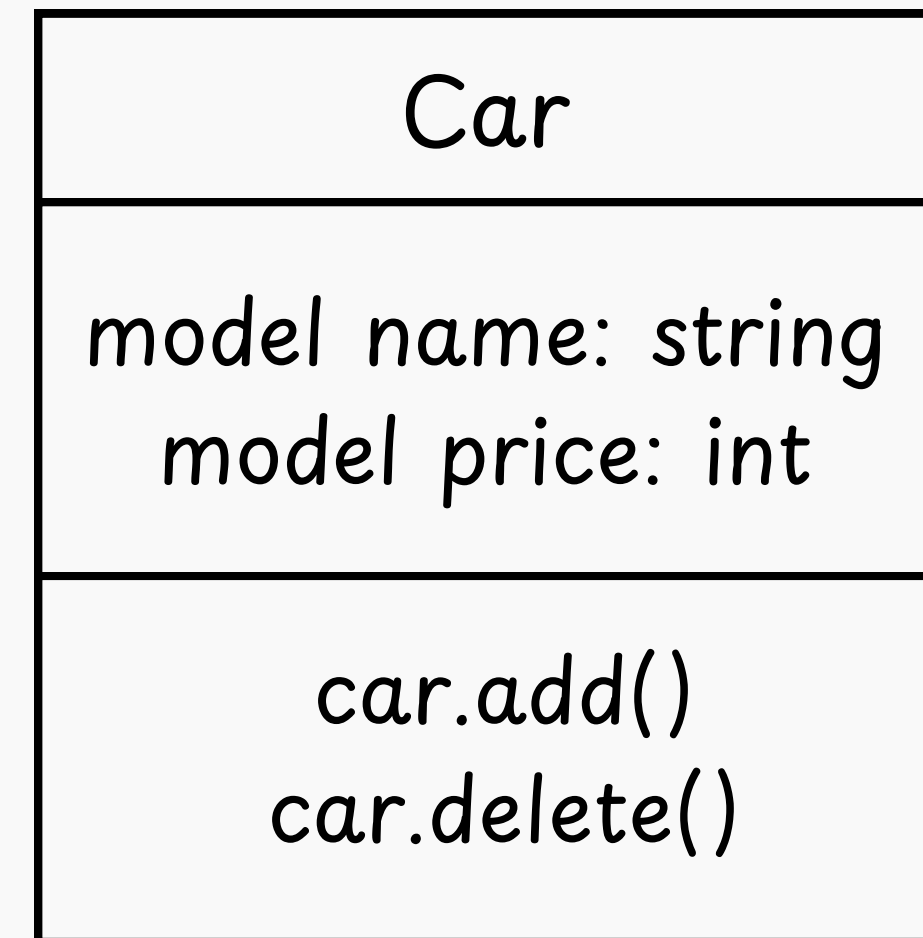
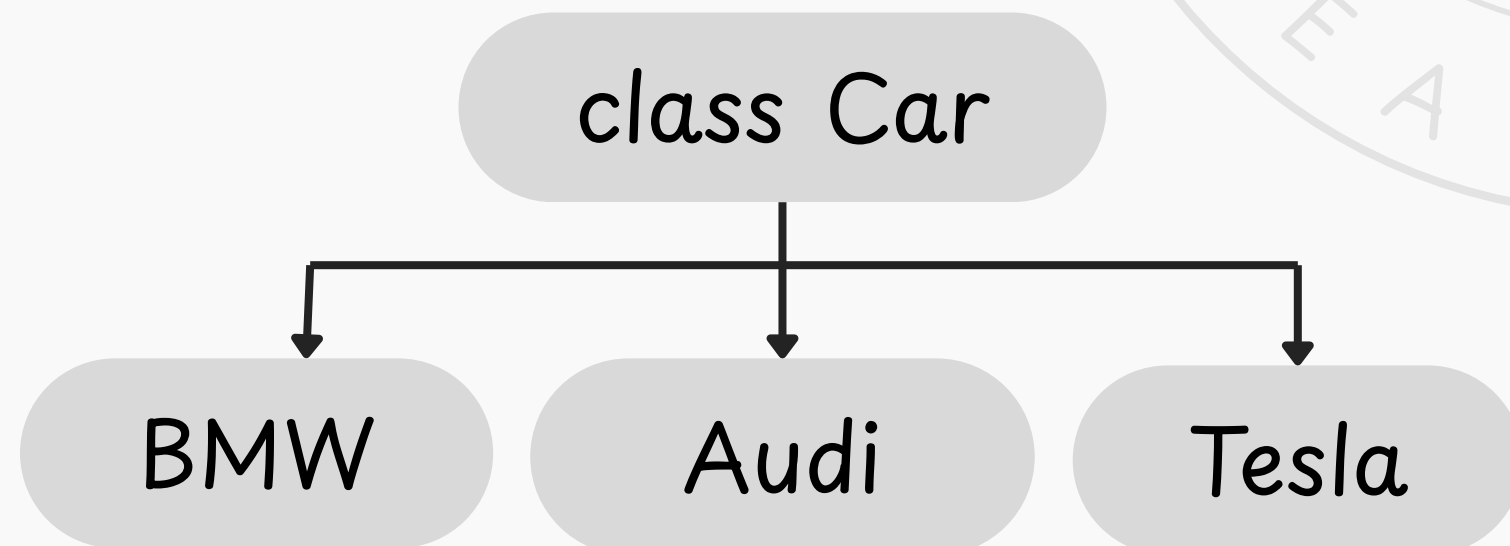
Pillars of Object Oriented Programming

1. Classes

A class is a blueprint or template used to create objects. It defines the data members (variables) and functions (methods) that an object will have.

Example:

- Class = Car
- Objects = BMW, Audi, Tesla

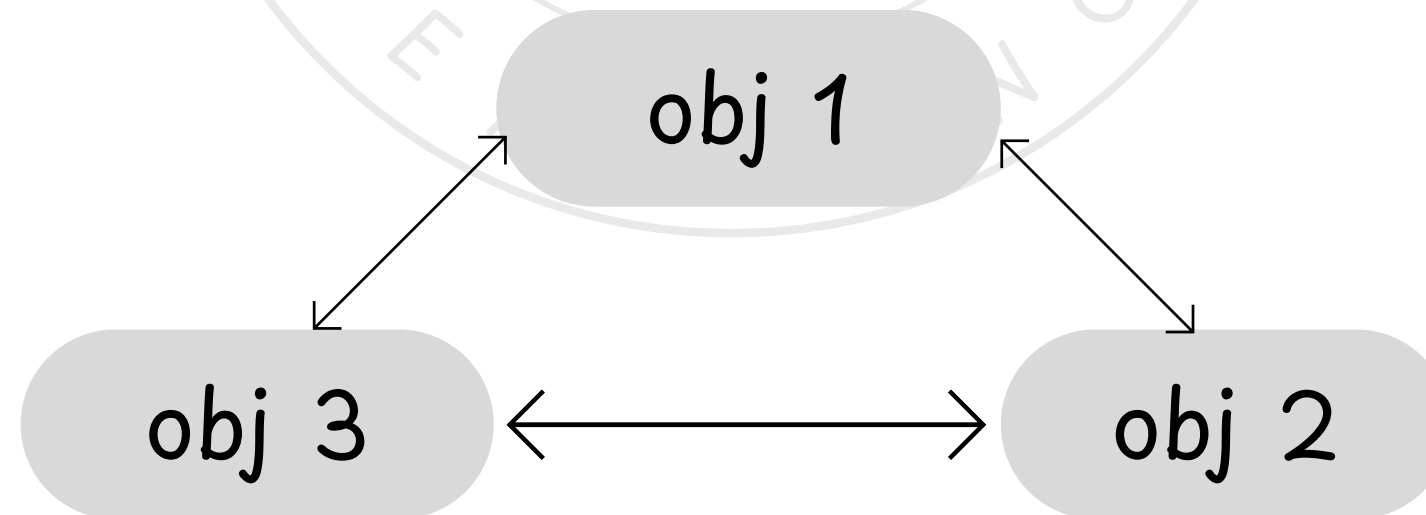


class diagram

Pillars of Object Oriented Programming

2. Objects

- An object is an instance of a class. It is a real entity that has data and behavior.
- Every object has a unique identity or memory location.
- During the execution of a programs objects interact with each others
- The state of an object is represented by its data values or attributes.
- Consists of **attributes(data)** and **method(functions)**



Pillars of Object Oriented Programming

Class	Object
A class is a blueprint or template.	An object is an instance of a class.
Class does not occupy memory when declared.	Object occupies memory when created.
Class defines data members and functions.	Object uses the properties and functions of the class.
A class is logical in nature.	An object is physical in nature.
Example: Student	Example: s1, s2

Pillars of Object Oriented Programming

Object Identity

- Object identity means every object has its own unique existence and memory location, even if two objects contain the same data.
- Each object is different from another object.
- Identity helps the system recognize objects separately.
- Two objects may have same values but still remain different objects.

Pillars of Object Oriented Programming

3. Abstraction

- Abstraction is the process of showing only essential information and **hiding unnecessary implementation** details from the user.
- Abstraction means hiding internal details and showing only important features.
- Can be implemented using Interface and Abstract Class

Example:

When driving a car, we use:

- Steering
- Brake
- Accelerator

We do not need to know how the engine works internally.

Pillars of Object Oriented Programming

4. Encapsulation (information hiding)

- Encapsulation is the process of wrapping data and functions together into a single unit called a class.
- Encapsulation means binding data members and member functions together and protecting data from unauthorised access.
- Can implement using access modifiers (Public, Private & Protected)

Example:

A capsule contains medicines wrapped inside a single cover.
Similarly, data and methods are wrapped together inside a class.

Pillars of Object Oriented Programming

5. Inheritance

- Inheritance is the process in which **one class acquires** the properties and functions of **another class**.
- Inheritance allows a new class to use the features of an existing class
- Existing class properties can be **reused in another class**.
- New features can be added easily using derived classes.
- Changes in base class automatically apply to derived class.
- Duplicate code is minimized.

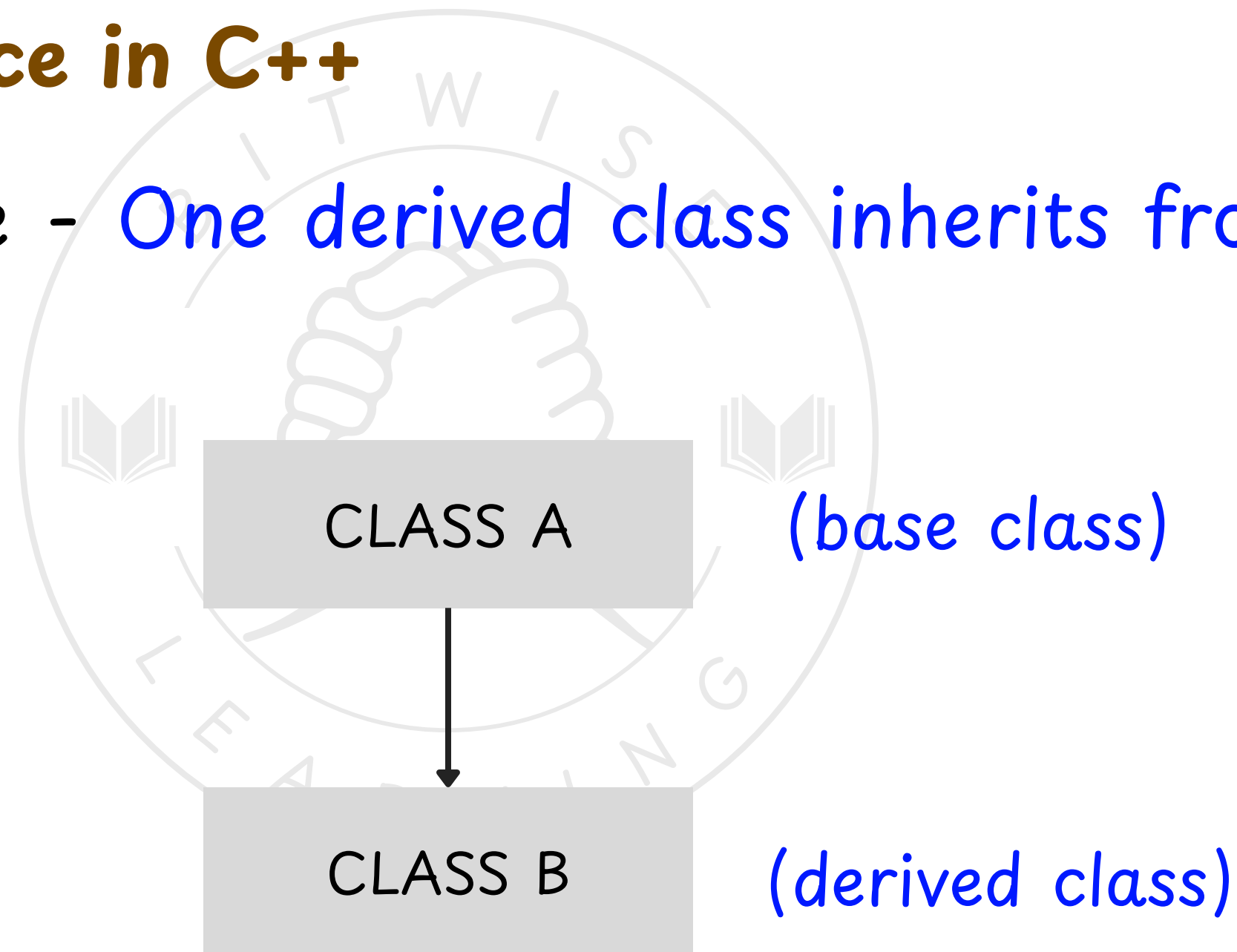
Example:

- Child inherits properties from parents.
- Similarly, a derived class inherits properties from a base class.

Pillars of Object Oriented Programming

Types of Inheritance in C++

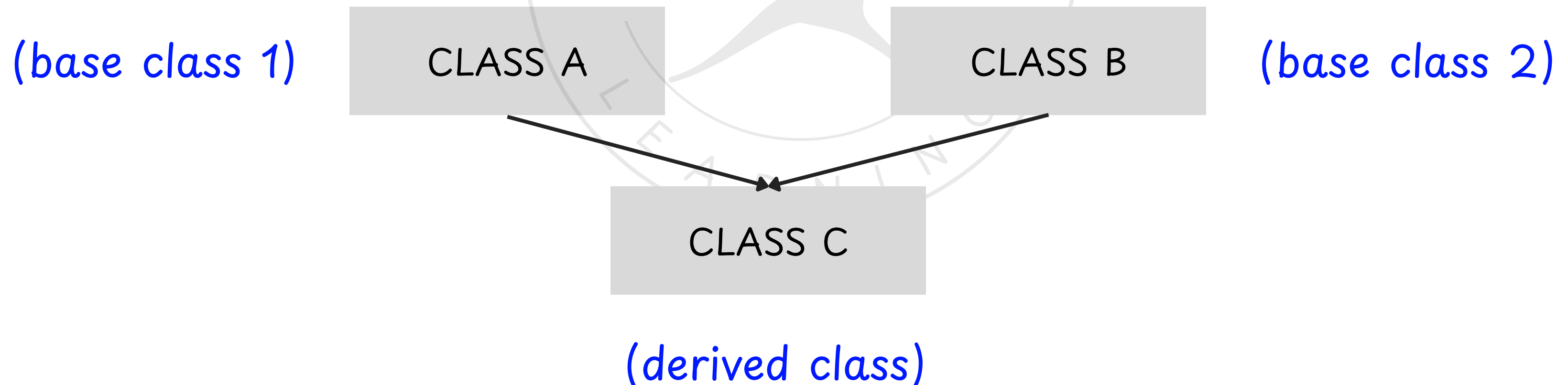
1. Single Inheritance - One derived class inherits from one base class.



Pillars of Object Oriented Programming

Types of Inheritance in C++

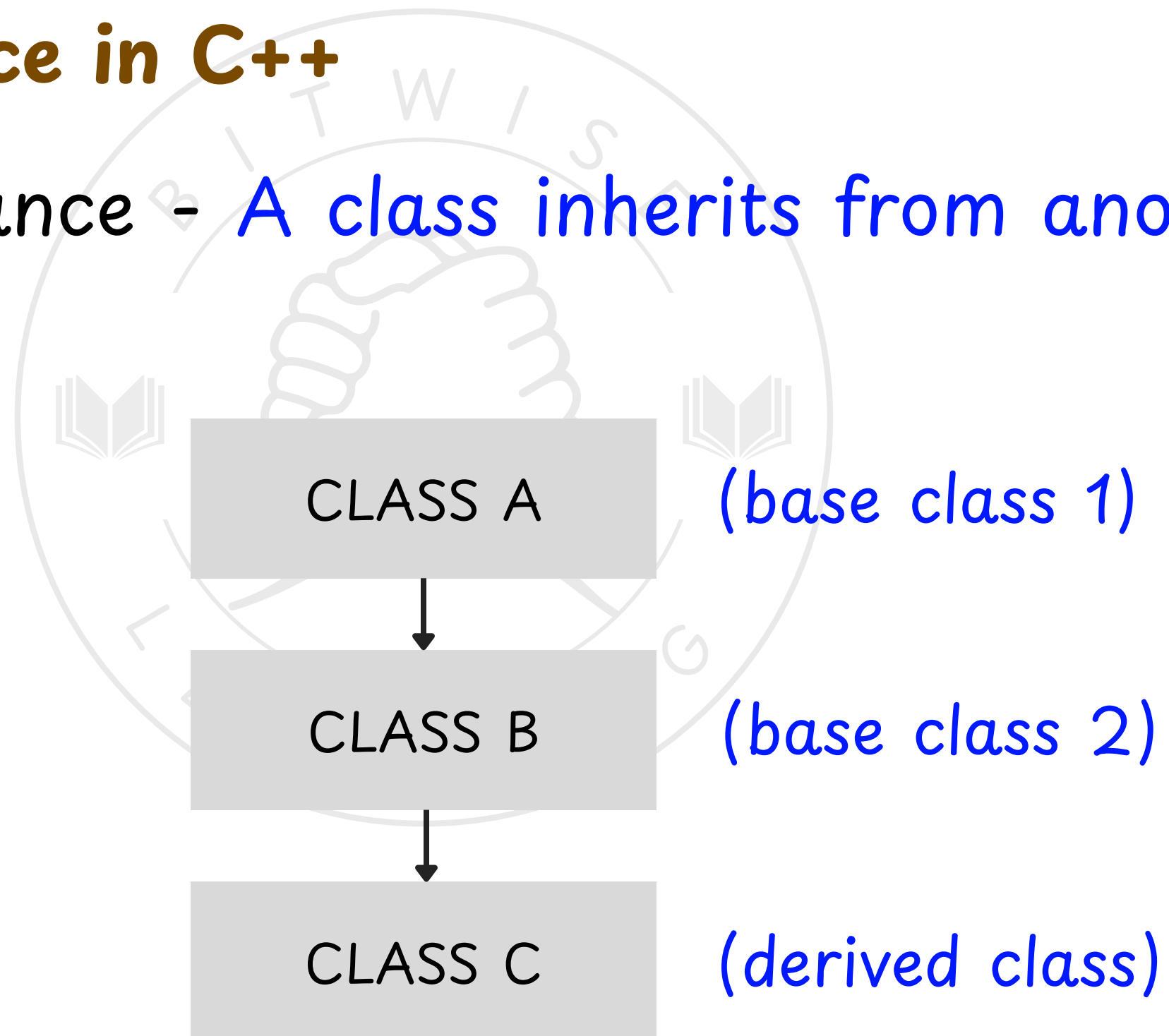
2. Multiple Inheritance - One derived class inherits from more than one base class.



Pillars of Object Oriented Programming

Types of Inheritance in C++

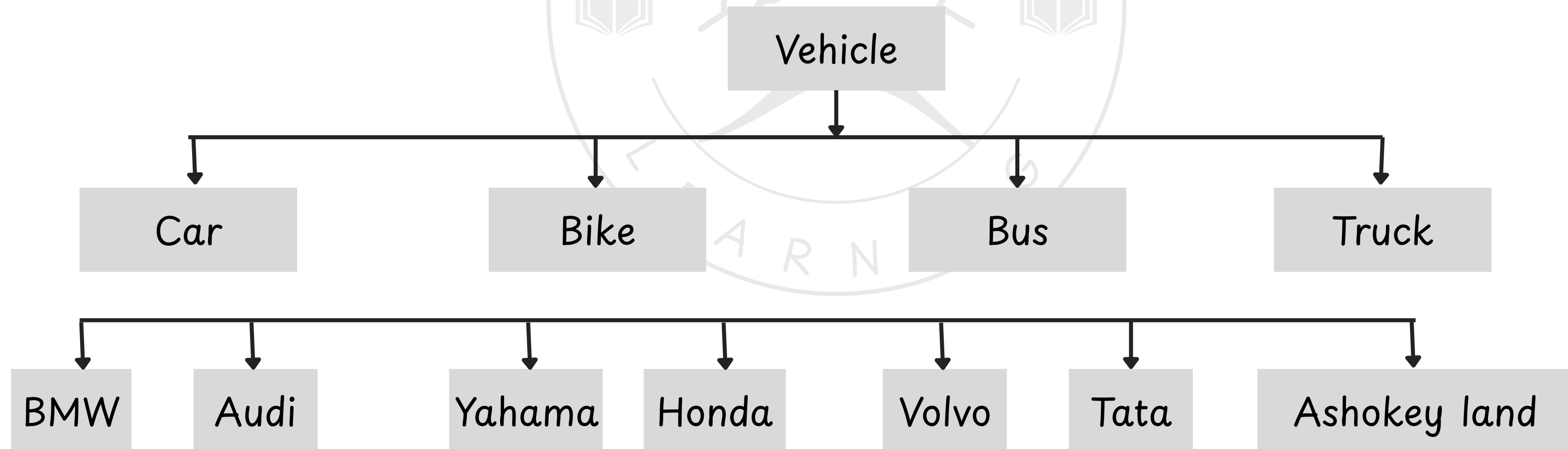
3. Multilevel Inheritance - A class inherits from another derived class.



Pillars of Object Oriented Programming

Types of Inheritance in C++

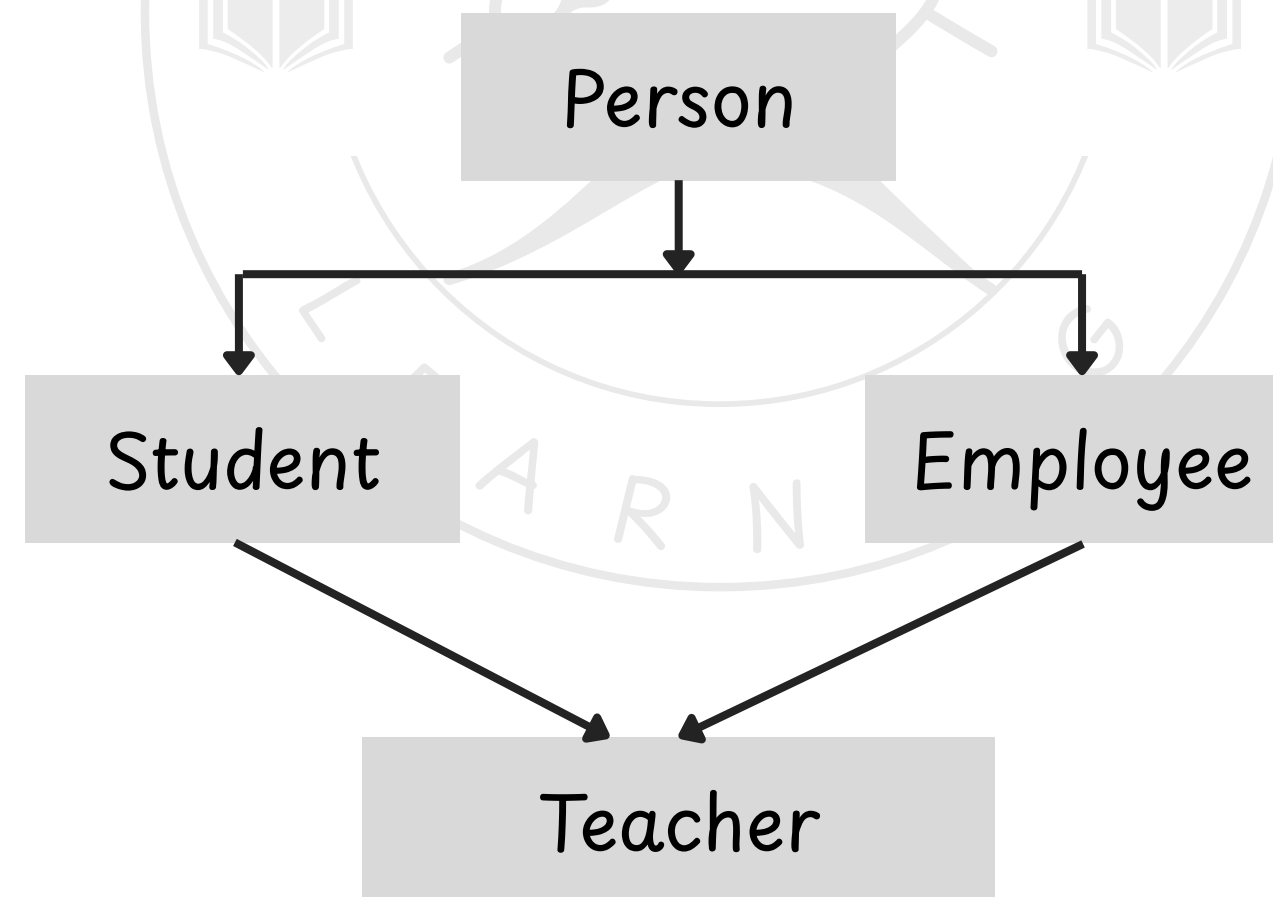
4. Hierarchical Inheritance - Multiple derived classes inherit from one base class.



Pillars of Object Oriented Programming

Types of Inheritance in C++

5. Hybrid Inheritance - Combination of two or more types of inheritance.



Pillars of Object Oriented Programming

6. Polymorphism

- Polymorphism is the ability of a function, object, or operator to **behave in different ways** in different situations.
- It helps in performing one task in many forms.
- It focuses on increasing program flexibility.
- Improving code reusability.

Example:

A mobile phone can be used for:

- Calling
- Gaming
- Watching videos

One device performs many functions, so this is an example of Polymorphism.

Generosity (Genericity)

- Generosity refers to the powerful ability of classes, objects, and functions to work smoothly with completely different data types without requiring explicit type casting.
- This feature makes code highly reusable and flexible, significantly reducing the developer's workload by preventing the need to write separate functions for every single data type.

Modelling

What is Model

- A model is a simplified representation of a real-world object, system, or concept used to understand, design, or explain how something works.
- A model represents the structure or working of a real system in a simplified form.

Example:

- A globe is a model of the Earth.
- In software engineering, a class diagram is a model used to represent the structure of classes and objects in a system.

Modelling

Principles of modelling

1. Choice of Model Matters:

The selected model affects how the problem is understood and solved.

2. Different Models for Different Purposes

Different types of models are used for different views of a system.

3. Models Should Be Simple and Understandable

A good model should clearly represent the system without unnecessary complexity.

4. Models Help in Visualizing the System

Models provide a clear structure and design of the system before implementation.

5. Every Model Must Be Accurate

The model should correctly represent real-world requirements and system behavior.

Modelling

Object Oriented Modelling

- Object-Oriented Modelling is the process of designing a system using objects, classes, and their relationships.
- It represents real-world entities in the form of objects.

Example:

In a school management system:

- Student, Teacher, and Classroom are treated as objects.
- Their properties and relationships are represented using object-oriented modelling.

UML

Introduction to UML

- UML (Unified Modeling Language) is the standard language used to specify, visualize, construct, and document the artifacts of a software system.
- UML was created by the Object Management Group (OMG) in January 1997.
- It is not a programming language. It is a visual modeling language.
However, modern developer tools can automatically generate actual code (like Java or C++) by analyzing UML diagrams.
- While mostly used for software, UML is flexible enough to model non-software systems, such as the process flow in a manufacturing plant.

UML

Introduction to UML

It have 4 core functions:

- Visualizing: Uses well-defined, **unambiguous semantic notations** so any developer can understand the blueprint.
- Specifying: Builds models that are precise, complete, and unambiguous.
- Constructing: Connects visual models directly to object-oriented programming languages.
- Documenting: Records all requirements, architecture, project plans, and release data.

UML

Conceptual Model of the UML

To deeply understand UML, you must understand its Conceptual Model, which consists of three major elements: **Basic Building Blocks**, **Rules**, and **Common Mechanisms**.

Basic Building Blocks

The building blocks are broken down into three categories: **Things**, **Relationships**, and **Diagrams**

UML

Things (Elements)

These are further of 4 types

1. Structural Things

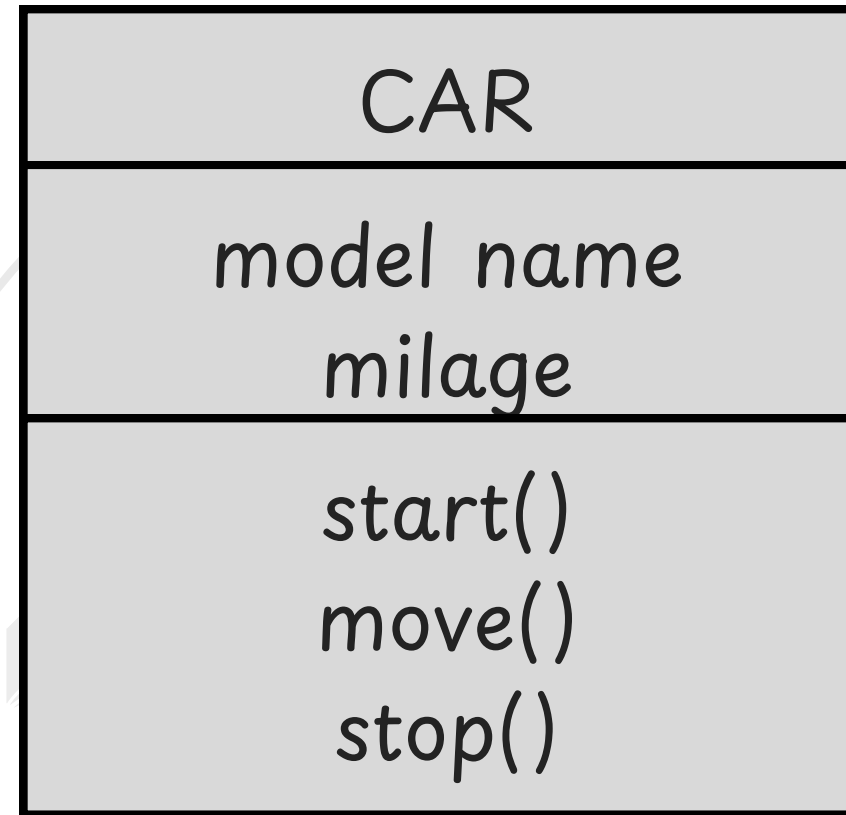
They are nouns of UML and mostly are static part of a model. They are also 7 in numbers

- **Class:**

A class is a blueprint or template **used to create objects**. It defines the data members and member functions of an object. Classes help in:

- Organizing data and functions together
- Creating multiple objects easily
- Improving code reusability

UML

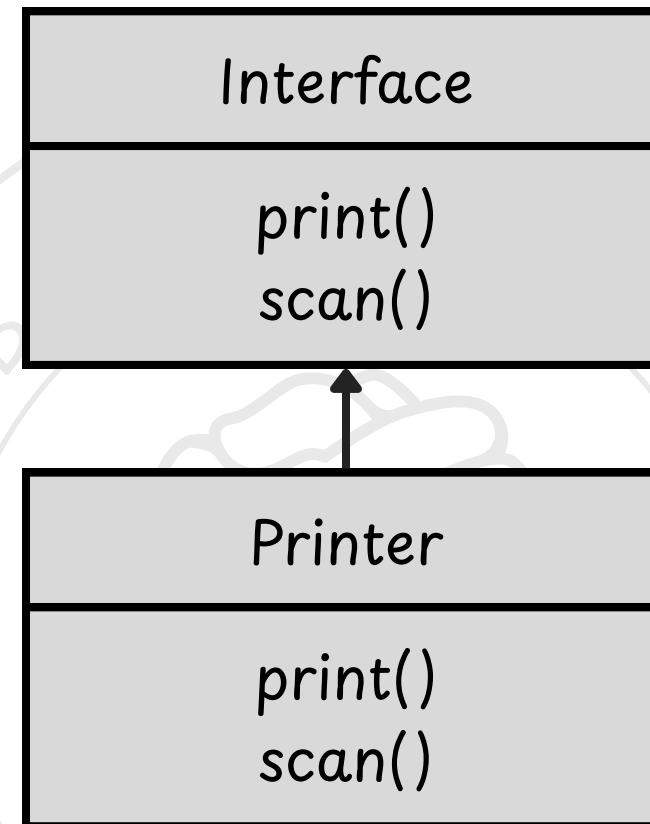


- **Interface:**

An interface is a collection of abstract functions that defines **what a class should do**, without providing implementation details. Interfaces help in:

- Achieving abstraction
- Improving flexibility
- Supporting multiple inheritance

UML

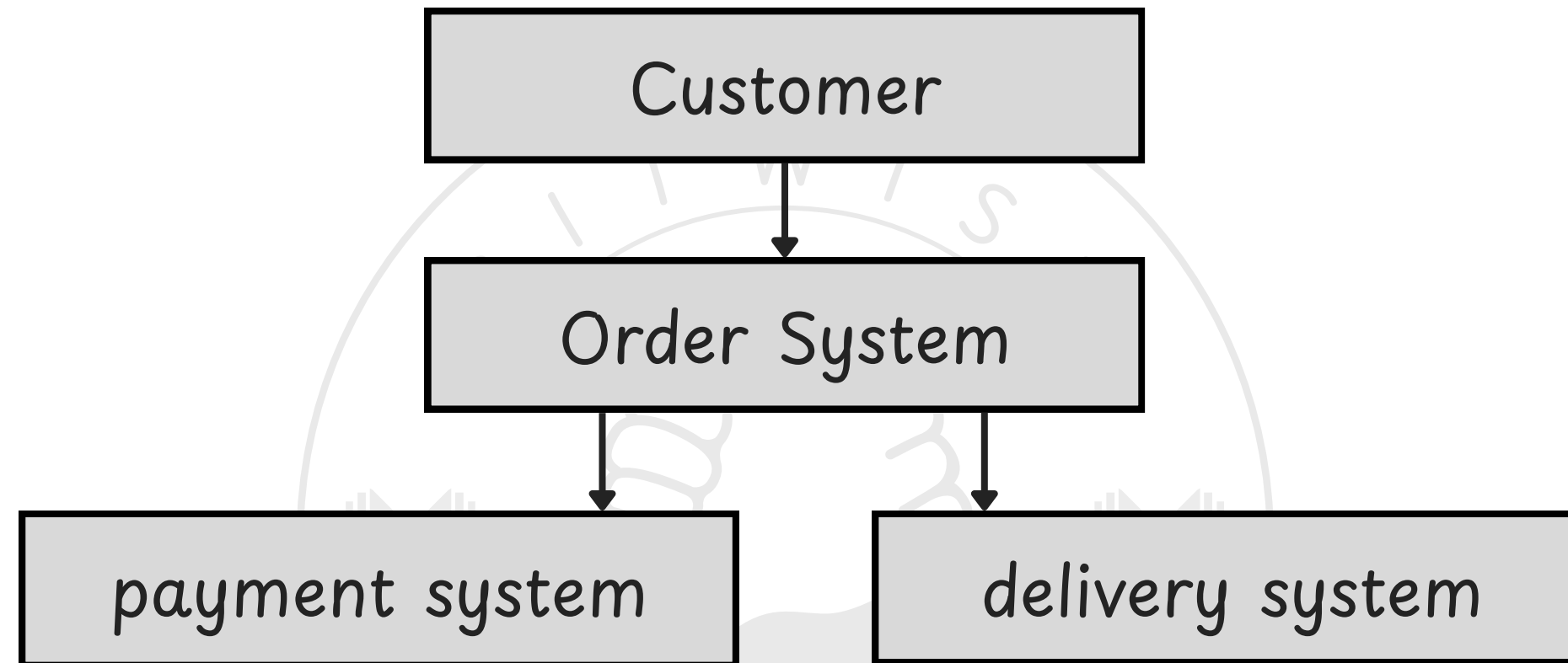


- **collaboration**

Collaboration represents interaction between objects and classes to complete a particular task. It helps in:

- Coordinating object communication
- Dividing tasks among objects
- Simplifying system behavior

UML

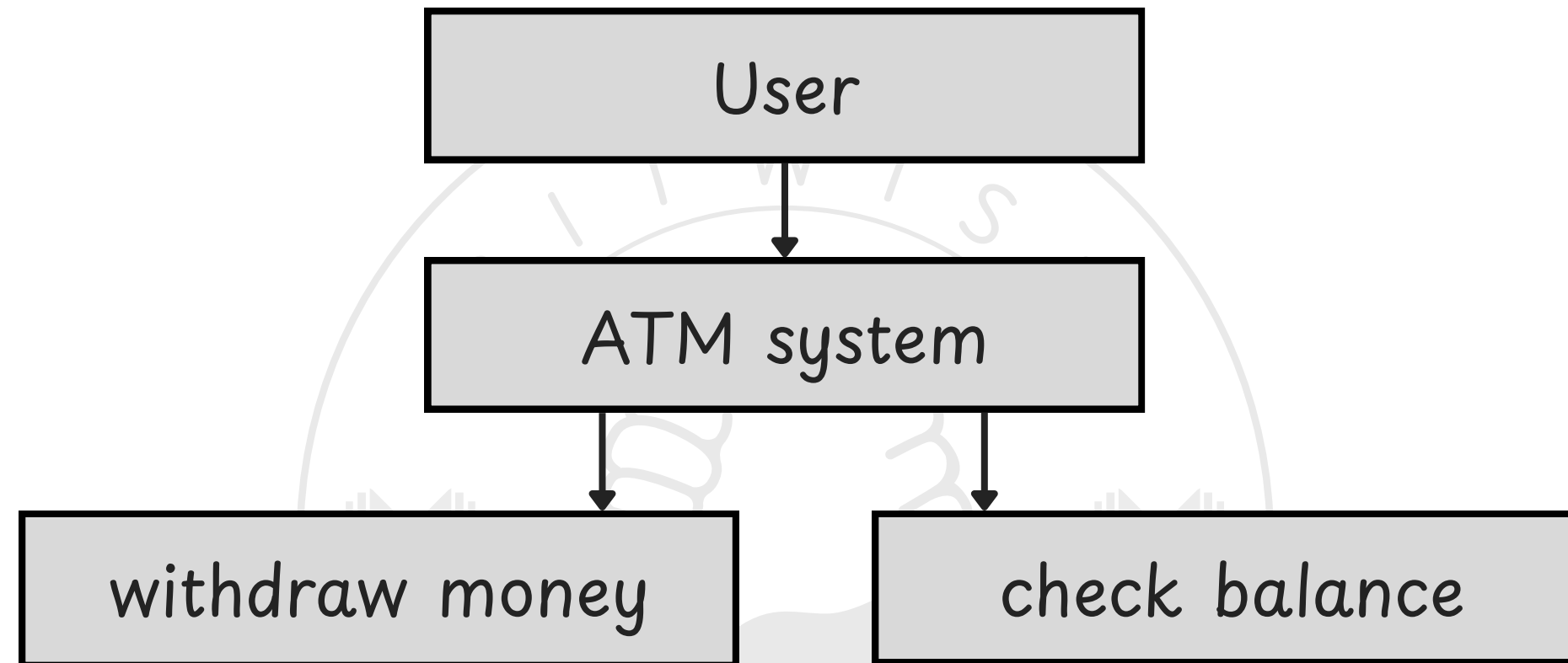


- **use case**

A use case describes the interaction between a user and the system to achieve a specific goal. It helps in:

- Understanding system requirements
- Identifying system functionality
- Improving system analysis

UML

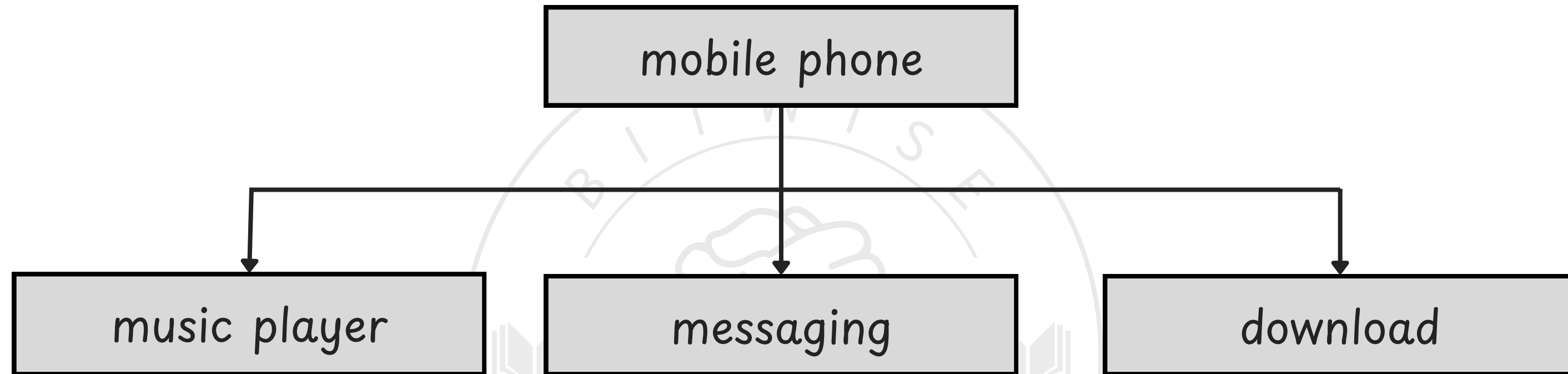


- **active class**

An active class is a class whose objects operate independently and may run simultaneously using their own thread or process. It helps in:

- Supporting multitasking
- Improving parallel processing
- Managing independent activities

UML

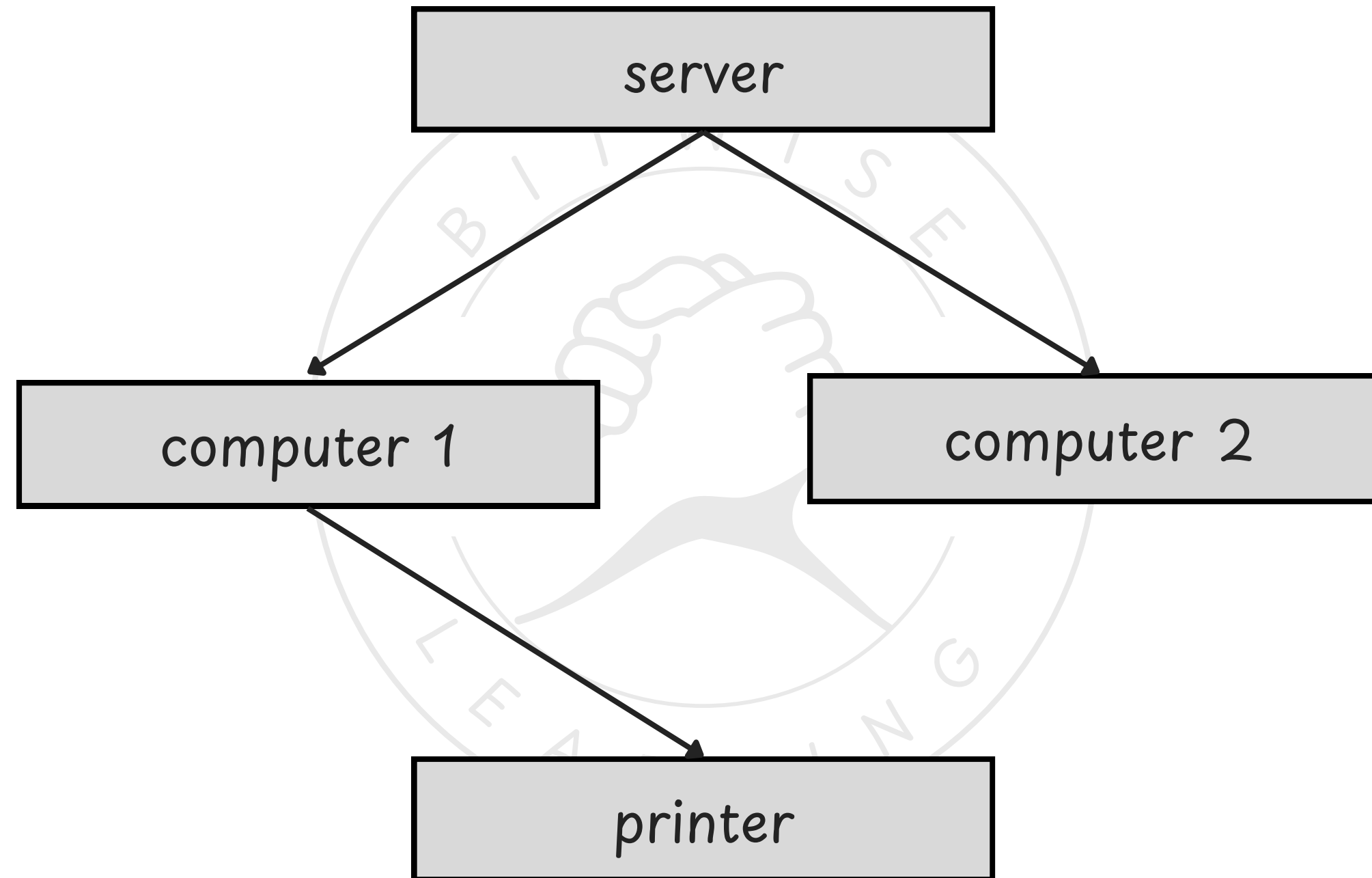


- **component**

A component is a physical and replaceable part of a system that provides implementation of a set of functions. Components help in:

- Dividing large systems into modules
- Improving maintainability
- Supporting software reuse

UML



UML

2. Behavioural Things

Behavioural things represent the dynamic behavior of a system and how objects interact with each other.

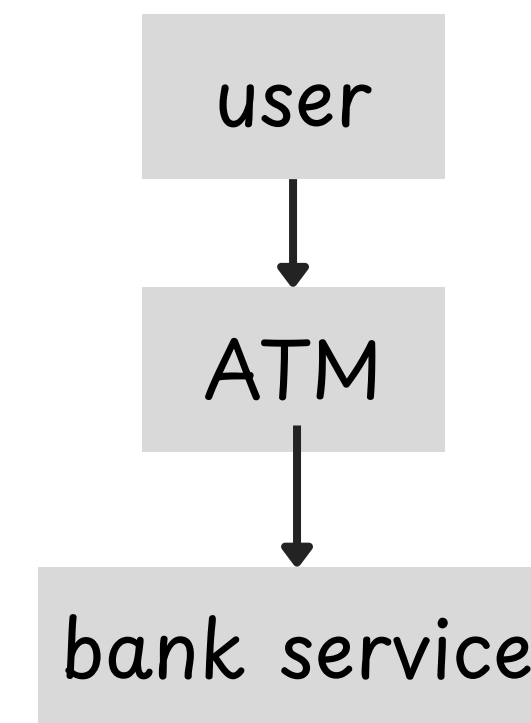
- **Interaction**

Interaction shows the exchange of messages between objects in a system.

Example:

In an ATM system:

- User requests withdrawal
- ATM verifies PIN
- Bank server processes transaction



UML

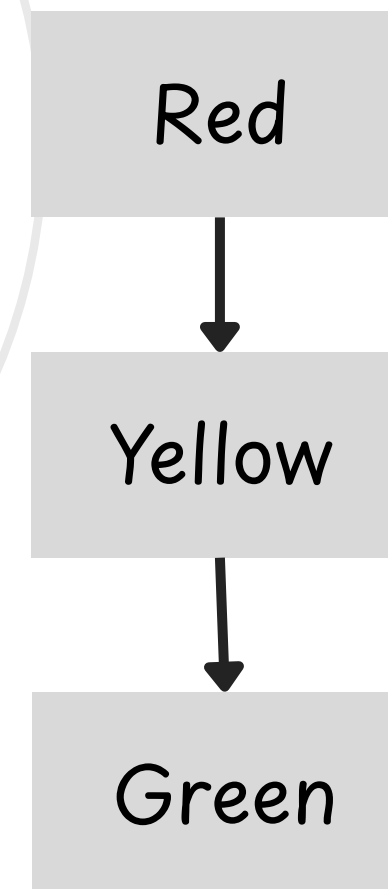
- **State Machine**

A state machine represents the different states of an object during its life cycle..

Example:

A traffic light changes states:

- Red
- Yellow
- Green



UML

3. Grouping Things

Grouping things are the mechanisms used to organize UML elements into groups or packages.

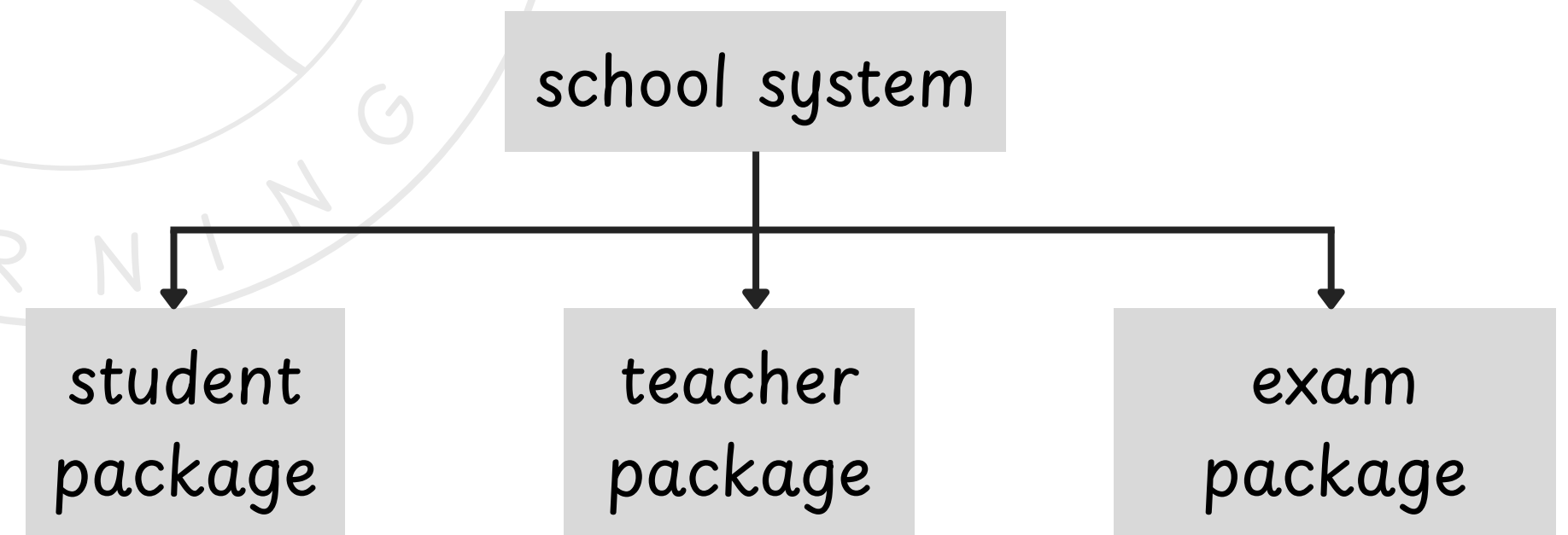
Main Grouping Thing

Package: A package is used to group related classes, objects, and other UML elements together.

Example:

In a school management system:

- Student package
- Teacher package
- Exam package



UML

4. Annotation Things

Annotation things are explanatory notes or comments added to UML diagrams to provide extra information about a model element.

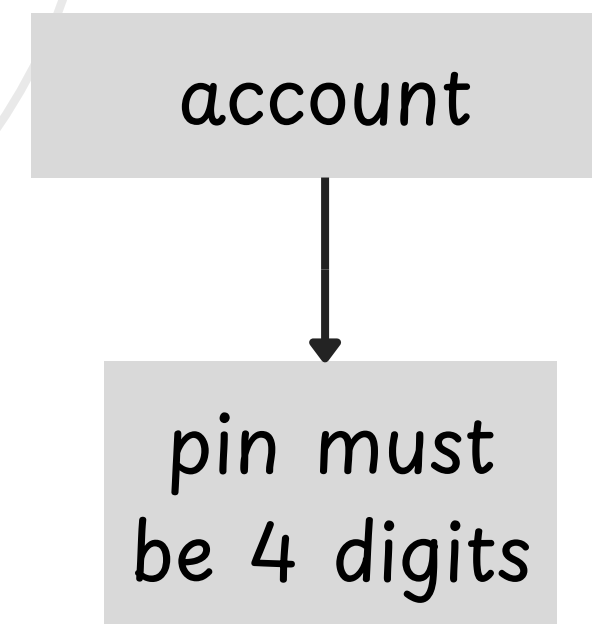
Main Annotation Thing

Node: A note is used to attach comments or explanations to UML elements.

Example:

In a banking system diagram, a note may specify:

- “PIN must be 4 digits”



UML

Relationship

These are further of 4 types

1. Dependency

Dependency is a relationship in which one class temporarily uses another class. Changes in one class may affect the dependent class. It helps in:

- Showing temporary usage
- Representing weak relationships
- Defining service dependency



UML

2. Association

Association represents a connection between two classes where objects communicate with each other. It helps in:

- Showing object relationships
- Representing communication
- Defining structural connection

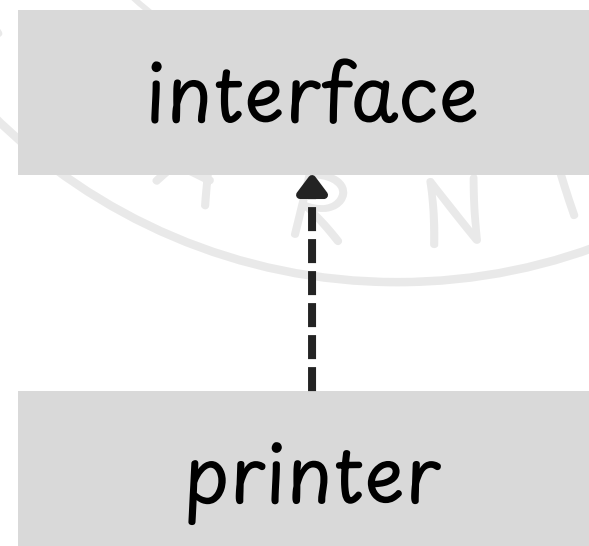


UML

3. Realization

Realization is a relationship in which a class implements an interface. It helps in:

- Achieving abstraction
- Defining common behavior
- Supporting interface implementation



UML

4. Aggregation

Aggregation is a weak “**has-a**” relationship where contained objects can exist independently. It helps in:

- Representing ownership
- Improving modularity
- Supporting object independence



UML

Diagram

UML (Unified Modeling Language) diagrams are graphical representations used to visualize, design, and document a software system.

1. Class Diagram

- Class diagram represents classes, attributes, methods, and relationships in a system.
- It helps in designing the static structure of a program.

2. Object Diagram

- Object diagram represents objects and their relationships at a specific point in time.
- It shows the practical view of class diagrams.

UML

3. Use Case Diagram

- Use case diagram represents interaction between users and the system.
- It helps in understanding system functionality and user requirements.

4. Sequence Diagram

- Sequence diagram shows the order of message exchange between objects over time.
- It explains how objects communicate step by step.

5. Collaboration Diagram

- Collaboration diagram represents communication and interaction among objects to perform a task.

UML

6. State Chart Diagram

- State chart diagram represents different states of an object and how it changes from one state to another.

7. Activity Diagram

- Activity diagram represents the flow of activities or operations in a system. It is similar to a flowchart.

8. Component Diagram

- Component diagram represents physical software components and their relationships within a system.

9. Deployment Diagram

- Deployment diagram represents hardware devices and deployment of software components in a network or system.

UML

Rules of UML

Names: Every element must have a distinct name.

Scope: The exact context and meaning of the name must be defined.

Visibility: Defines how those names can be seen and used by others.

Integrity: Ensures things are properly and consistently related to one another.

Execution: Defines how the model will run or simulate dynamically.

Common Mechanisms

Specification: Providing textual statements and syntax behind the graphical models.

Adornment: Using unique, unambiguous graphical notations.

Common Division: Breaking the model down into smaller, modular divisions.

Extensibility Mechanisms: Allowing users to extend UML using Stereotypes (extending vocabulary), Tagged Values (extending properties), and Constraints (modifying semantic rules).

UML

Architecture of UML

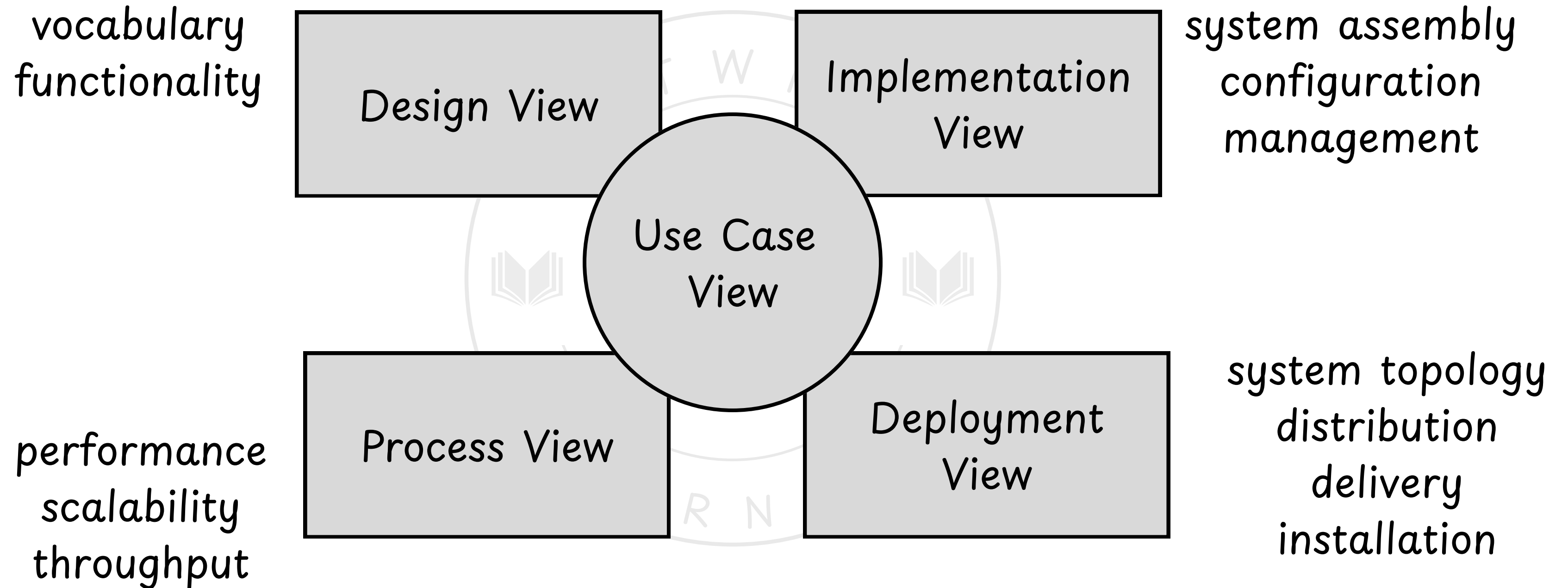
- Use Case View (The "+1" Center): This sits at the center of the architecture and connects all the other views. It represents the core functionality of the system from the perspective of an end-user.
- Design View: The static perspective used by designers. It encompasses classes, interfaces, and collaborations to form the basic "vocabulary" of the problem.
- Process View: The dynamic perspective. It addresses system concurrency, threads, performance, and synchronization (how the system flows and functions simultaneously).

UML

Architecture of UML

- Implementation View: Used by programmers. It defines how components are physically assembled together to create the system (represented heavily by Component Diagrams).
- Deployment View: Used by system administrators. It shows the physical hardware topology, addressing how the system will be distributed, delivered, and installed on actual nodes/servers.

UML



Difference between POP & OOP

Basis of Comparison	POP	OOP
Approach	Follows a Top-Down approach.	Follows a Bottom-Up approach.
Division of Program	The entire program is divided into small parts called Functions	The program is divided into small parts called Objects and Classes
Security (Data Hiding)	Less secure because data hiding is not possible; data is publicly available to all functions.	Highly secure because it uses Encapsulation and Abstraction to hide confidential data.
Access Specifiers	Does not have any access specifiers	Has strict access modifiers like Public, Private, and Protected to control who sees the data

Difference between POP & OOP

Basis of Comparison	POP	OOP
Memory Usage	Takes very less memory	Takes more memory because objects occupy physical space.
Focus	Focuses mainly on the Algorithm or the steps to do a task.	Focuses entirely on the Data and its security.
Examples	C, Fortran, and COBOL	C++, Java, and Python

Thank You